JavaScript: objetos literais

propriedades • acesso • alteração • objetos aninhados • arrays de objetos • for...in

Prof. Dionizio Porfírio
Unichristus • 2026.2

Objetivos de aprendizagem

Ao final da aula, o estudante deverá ser capaz de:

- Compreender objetos como estruturas que agrupam dados relacionados.
- Criar objetos utilizando a sintaxe literal { }.
- Acessar propriedades com notação de ponto e colchetes.
- Adicionar, alterar e remover propriedades.
- Compreender o comportamento de objetos declarados com const.
- Trabalhar com objetos aninhados e arrays de objetos.
- Percorrer propriedades com for...in e diferenciar seu uso de for...of.
- Aplicar find(), filter() e map() em arrays de objetos.

Por que usar objetos?

Quando vários valores descrevem a mesma entidade, um objeto permite agrupá-los com nomes que indicam o significado de cada dado.

Dados relacionados

Nome, preço e estoque podem representar um único produto.

Propriedades nomeadas

Em vez de depender apenas de posições, cada informação recebe uma chave.

Modelagem simples

Objetos ajudam a representar alunos, produtos, pedidos, livros e outros registros.

Um objeto literal é uma forma direta de representar uma entidade sem precisar criar uma classe.

Criando um objeto literal

Um objeto literal é escrito entre chaves. Cada propriedade é formada por uma chave e um valor.

```
let produto = {
    codigo: 10,
    nome: "Mouse",
    preco: 79.90,
    estoque: 5
};
```

Chaves

codigo, nome, preco e estoque identificam as propriedades.

Valores

Cada propriedade pode armazenar valores de tipos diferentes.

Separação

As propriedades são separadas por vírgulas dentro do objeto.

Acessando propriedades com ponto

A notação de ponto é a forma mais comum quando conhecemos diretamente o nome da propriedade.

Objeto

```
let aluno = {
    nome: "Ana",
    nota: 8.5
};

console.log(aluno.nome);
console.log(aluno.nota);
```

```
aluno identifica a estrutura completa.
```

Propriedade

aluno.nome acessa somente o valor associado à chave nome.

```
Leitura
```

A expressão pode ser usada em cálculos, condições ou mensagens.

Acessando propriedades com colchetes

Os colchetes permitem informar o nome da propriedade por meio de uma string ou de uma variável.

```
let produto = { nome: "Teclado", preco: 120 };
let campo = "preco";

console.log(produto["nome"]);
console.log(produto[campo]);
```

String

produto["nome"] acessa uma propriedade pelo texto da chave.

Variável

produto[campo] usa o conteúdo da variável para decidir qual propriedade acessar.

Essa forma é necessária quando o nome da propriedade é determinado durante a execução.

Ponto ou colchetes?

As duas formas acessam propriedades, mas não são intercambiáveis em todas as situações.

Notação de ponto

Use quando o nome da propriedade é conhecido e possui um identificador simples.
Ex.: aluno.nome

Colchetes

Use quando a chave vem de uma variável ou contém caracteres que não funcionam na notação de ponto.
Ex.: aluno[campo]

Chave dinâmica

```
let campo = prompt("Campo:");
console.log(aluno[campo]);
```

Leitura prática

Ponto: nome fixo no código.
Colchetes: nome decidido como dado.

Exemplo — cadastro de um produto

Crie um objeto produto a partir de informações informadas pelo usuário e depois apresente uma frase com seus dados.

- Solicite código, nome, preço e quantidade em estoque.
- Faça as conversões numéricas necessárias.
- Monte um único objeto literal com as quatro informações.
- Exiba nome, preço e estoque utilizando propriedades do objeto.
- Não utilize quatro arrays separados para representar o mesmo produto.

O objetivo é perceber a diferença entre guardar dados isolados e representar uma entidade completa em um único objeto.

Adicionar, alterar e remover propriedades

Objetos podem ter seu conteúdo modificado depois da criação.

```
let usuario = { nome: "Ana", idade: 20 };

usuario.idade = 21;
usuario.email = "ana@email.com";
delete usuario.email;
```

Alterar

Atribuir a uma propriedade existente substitui seu valor.

Adicionar e remover

Uma nova chave pode ser criada por atribuição; delete remove uma propriedade existente.

Objetos podem mudar de conteúdo ao longo da execução do programa.

const não torna o objeto imutável

Quando um objeto é declarado com const, a variável não pode receber outro objeto, mas suas propriedades ainda podem mudar.

```
const conta = {
    titular: "Bia",
    saldo: 100
};

conta.saldo += 50;
console.log(conta.saldo); // 150
```

Permitido

Alterar uma propriedade do mesmo objeto continua sendo possível.

Não permitido

conta = { titular: "Caio" } tentaria substituir a referência armazenada em conta.

Objetos podem conter outros objetos

Uma propriedade também pode armazenar outro objeto, permitindo representar estruturas com níveis.

```
let aluno = {
    nome: "Ana",
    endereco: {
        cidade: "Fortaleza",
        uf: "CE"
    }
};
```

Objeto externo

aluno representa a entidade principal.

Objeto interno

endereco agrupa informações que pertencem ao mesmo conjunto.

Acessando dados aninhados

Para chegar a uma propriedade interna, seguimos os níveis do objeto na ordem em que eles aparecem.

```
console.log(aluno.endereco.cidade);

aluno.endereco.cidade = "Quixadá";

console.log(aluno.endereco.cidade);
```

Leitura

aluno.endereco.cidade percorre duas propriedades até chegar ao valor.

Alteração

A mesma cadeia pode aparecer do lado esquerdo de uma atribuição.

Arrays de objetos

Quando precisamos armazenar várias entidades com a mesma estrutura, cada posição do array pode guardar um objeto.

```
let produtos = [
    { codigo: 1, nome: "Mouse", estoque: 8 },
    { codigo: 2, nome: "Teclado", estoque: 3 },
    { codigo: 3, nome: "Monitor", estoque: 5 }
];
```

Array

produtos reúne vários registros.

Objeto por posição

Cada elemento possui suas próprias propriedades.

Percorrendo um array de objetos

for...of percorre os elementos do array. Como cada elemento é um objeto, suas propriedades podem ser acessadas normalmente.

```
for (let produto of produtos) {
    console.log(`${produto.nome}:
${produto.estoque}`);
}
```

produto

A cada repetição, recebe um dos objetos armazenados no array.

Propriedades

produto.nome e produto.estoque acessam os dados do elemento atual.

for...in — percorrendo propriedades

for...in percorre os nomes das propriedades enumeráveis de um objeto.

```
let pessoa = { nome: "Ana", idade: 20, cidade:
"Fortaleza" };

for (let chave in pessoa) {
    console.log(chave, pessoa[chave]);
}
```

Chave

chave recebe "nome", depois "idade", depois "cidade".

Valor

pessoa[chave] acessa o valor associado à propriedade atual.

for...of e for...in têm objetivos diferentes

Os nomes são parecidos, mas cada estrutura percorre uma coisa diferente.

```
for (let produto of produtos) {
    console.log(produto.nome);
}

for (let chave in produto) {
    console.log(chave, produto[chave]);
}
```

for...of

Percorre os objetos armazenados no array.

for...in

Percorre os nomes das propriedades do objeto atual.

Exemplo — localizar um produto

Implemente uma busca por código em um array de objetos, utilizando o método find() estudado na aula anterior.

- Considere um array de produtos com codigo, nome e preco.
- Solicite um código para pesquisa.
- Utilize find() para localizar o primeiro objeto com o código informado.
- Se encontrar, exiba nome e preço.
- Caso contrário, informe que o produto não foi encontrado.

A condição do callback agora consulta uma propriedade do objeto: produto.codigo === codigoBuscado.

filter() também funciona com objetos

O callback pode analisar propriedades do objeto para decidir quais elementos permanecerão no novo array.

```
let disponiveis = produtos.filter(produto => {
    return produto.estoque > 0;
});

console.log(disponiveis);
```

Seleção

Somente objetos cujo estoque seja maior que zero entram no resultado.

Novo array

filter() não remove elementos do array produtos.

map() pode extrair ou transformar propriedades

O callback pode receber um objeto e decidir qual informação será colocada no novo array.

Extrair

```
let nomes = produtos.map(p => p.nome);
```

Transformar

```
let etiquetas = produtos.map(p => `${p.nome} - ${p.estoque}`);
```

Resultado

map() devolve um novo array com um valor produzido para cada objeto original.

O array produtos continua armazenando os objetos completos.

Exemplo — processando um catálogo

Combine filter() e map() para descobrir quais produtos precisam de reposição e obter somente seus nomes.

1. Selecionar

```
let reposicao = produtos.filter(p => p.estoque < 3);
```

2. Transformar

```
let nomes = reposicao.map(p => p.nome);
```

Primeiro selecione os objetos; depois transforme o resultado. Faça em duas etapas antes de encadear métodos.

Objetos são valores por referência

Atribuir um objeto a outra variável não cria automaticamente uma cópia independente.

```
let original = { nome: "Ana", pontos: 10
};
let copia = original;

copia.pontos = 20;
console.log(original.pontos); // 20
```

```
console.log(original === copia);
// true
```

Mesma referência

As duas variáveis apontam para o mesmo objeto.

Consequência

Alterações feitas por uma referência podem ser observadas pela outra.

Erros comuns com objetos

Confundir objeto com array

produto.nome usa uma propriedade; produto[0] só faria sentido se existisse uma chave 0.

Usar ponto com uma chave variável

produto.campo procura literalmente a propriedade campo. Para uma variável, use produto[campo].

Achar que const congela propriedades

const impede reatribuir a variável, mas não impede modificar o objeto.

Usar for...in para percorrer valores do array

Em arrays, prefira for...of quando o objetivo for trabalhar com os elementos.

Esquecer o objeto atual no array

Em produtos[i].preco ou produto.preco, é preciso acessar a propriedade a partir do elemento.

Confundir cópia com referência

let copia = original faz as duas variáveis apontarem para o mesmo objeto.

Exercício 1 — atualização de pontuação

Desenvolva a função atualizarPontuacao(jogadores, apelido, pontos).

- Receba um array de objetos com apelido, pontos e ativo.
- Localize o jogador cujo apelido seja igual ao informado.
- A pontuação só poderá ser alterada se o jogador estiver ativo.
- Some pontos ao valor atual e retorne a nova pontuação.
- Retorne -1 quando o jogador não existir ou estiver inativo.

A função deverá alterar o objeto localizado no próprio array.

Exercício 2 — tarefas prioritárias

Desenvolva a função listarPrioritarias(tarefas, prioridadeMinima).

- Cada objeto possui descricao, prioridade e concluida.
- Utilize filter() para selecionar tarefas não concluídas com prioridade maior ou igual ao limite informado.
- Utilize map() para gerar um novo array contendo somente as descrições selecionadas.
- Não altere o array original.
- Retorne o novo array ao final.

Exemplo: uma tarefa { descricao: "Revisar prova", prioridade: 4, concluida: false } poderá entrar no resultado quando o limite for 3.

Resumo da aula

Objetos literais agrupam propriedades relacionadas e permitem representar entidades de forma direta.

Objeto literal

É criado com { } e armazena pares de chave e valor.

Acesso

Ponto é ideal para chaves conhecidas; colchetes permitem chaves dinâmicas.

Modificação

Propriedades podem ser adicionadas, alteradas e removidas.

Arrays de objetos

Permitem armazenar conjuntos de registros e reutilizar find(), filter() e map().

for...in

Percorre nomes de propriedades; para elementos de arrays, prefira for...of.

Próxima aula

JSON: diferença entre objeto JavaScript e texto JSON, JSON.stringify() e JSON.parse().